MATERIAL PARA ASISTENTES

Agentes de IA con Pydantic AI

Guía de la charla: lo que se explicó, por qué importa y cómo seguir aprendiendo.

Preparado para	Fecha y hora	Preparado por
PyCon Panamá 2026 · Python intermedio	Viernes 23 de octubre de 2026	Ricardo Tovar

01 — De qué trata esta guía

Esta guía acompaña la charla «Agentes de IA con Pydantic AI: de cero a producción con menos magia y más ingeniería». Reúne lo que se explicó en la sala, con más detalle del que cabe en una diapositiva, y propone un camino para seguir aprendiendo después.

La tesis de la charla cabe en una frase: **un agente útil no es solo un modelo bueno. Es un modelo dentro de un sistema que lo vuelve controlable.** Ese sistema se construye con piezas conocidas de Python: tipos, validación, funciones, un servicio web, límites y trazas.

Para quién está escrita: personas con Python intermedio que quieren pasar de un experimento con un modelo de lenguaje a un servicio que otras personas puedan usar con confianza. No hace falta experiencia previa con agentes.

02 — Lo que se vio en la charla

Parada	Qué se explicó
Agent loop	Cómo decide un agente
El stack	Las piezas de un servicio con agente
Un agente tipado	Salida, herramienta y dependencias con tipos
El harness	El sistema que rodea al modelo
FastAPI + agente	Una petición, de punta a punta
Producción	Operar con control bajo carga
Agentes de dominio	Mi mentalidad y por qué importa
Demostración	Un agente resolviendo un trabajo concreto

03 — La idea de partida

La frase con la que abre la charla viene de la documentación de Pydantic AI v1.0.0: «*We built Pydantic AI with one simple aim: to bring that FastAPI feeling to GenAI app and agent development*». En español: «Construimos Pydantic AI con un solo objetivo: llevar esa sensación de FastAPI al desarrollo de aplicaciones y agentes de IA generativa».

«Esa sensación de FastAPI» es la experiencia de usar funciones normales de Python con tipos: el framework valida por ti, el editor te ayuda con el autocompletado, los errores son claros y hay poco código de pegamento. La promesa es que armar un agente se sienta igual de directo que armar una API.

04 — Agent loop: cómo decide un agente

Un agente es un modelo que elige qué herramienta usar hasta tener una respuesta. El ciclo es este:

1. La persona usuaria hace una petición.
2. El modelo la lee y decide si puede responder o si necesita una herramienta.
3. Si necesita una, la elige y la llama con argumentos.
4. La herramienta ejecuta y devuelve un resultado, que se suma al contexto.
5. El modelo vuelve a decidir con ese nuevo contexto.
6. Cuando tiene suficiente, produce la respuesta, que se valida antes de entregarla.

Lo importante es que el modelo **decide**, pero el código **ejecuta**. El modelo no corre nada por su cuenta: pide llamar a una función que tú escribiste y tú controlas lo que esa función puede hacer.

05 — El stack: las piezas de un servicio con agente

La charla propone cuatro piezas y una capa que observa todo:

- **Cliente.** La aplicación o interfaz que envía la petición y muestra la salida.
- **API asíncrona (FastAPI).** Recibe la petición y la valida con Pydantic antes de que llegue al agente.
- **Agente tipado (Pydantic AI).** El modelo, las herramientas y una salida con tipos, construidos sobre Pydantic.
- **Datos (PostgreSQL).** El historial, los datos de la aplicación y, si hace falta, búsqueda vectorial con pgvector.
- **Observabilidad (Logfire).** Del equipo de Pydantic. Traza FastAPI, Pydantic, Pydantic AI y las consultas a Postgres.

La razón de usar estas piezas juntas es que comparten la misma base: los modelos de Pydantic. El mismo tipo que valida la petición describe la salida del agente y los argumentos de las herramientas.

06 — Un agente tipado

La salida, el agente y la herramienta caben en 17 líneas. Este es el código que se mostró, con datos sintéticos:

```
class Triage(BaseModel):
    category: Literal["facturacion", "acceso", "error", "otro"]
    urgent: bool
    needs_review: bool = Field(description="True si falta información")
    summary: str

agent = Agent(
    deps_type=Deps,
    output_type=Triage,
    instructions="Clasifica el ticket. Si falta información, marca needs_review.",
)

@agent.tool
async def customer_plan(ctx: RunContext[Deps], customer_id: int) -> str:
    """Devuelve el plan del cliente: free, pro o enterprise."""
    plan = await ctx.deps.db.customer_plan(customer_id)
    return plan or "desconocido"
```

Cómo leerlo, pieza por pieza:

- `Triage` **es el contrato de salida.** Es un modelo de Pydantic. El modelo de lenguaje no puede devolver cualquier cosa: la respuesta se valida contra estos campos. Si `category` no es una de las cuatro opciones, la validación falla.
- `Agent(...)` **define el agente.** `output_type=Triage` conecta el contrato; `deps_type=Deps` declara qué dependencias recibe (por ejemplo, el acceso a la base de datos); `instructions` es el texto que orienta al modelo.
- `@agent.tool` **expone una función.** La herramienta recibe un `RunContext` con las dependencias ya tipadas. Así no hay variables globales ni conexiones escondidas: lo que la herramienta necesita llega por parámetro.

El campo `needs_review` merece atención: es una salida honesta. En lugar de forzar al modelo a inventar cuando falta información, se le da una forma explícita de decir «aquí hace falta una persona».

07 — El harness: el sistema que rodea al modelo

Harness es el sistema que rodea al modelo y organiza contexto, agent loop, herramientas, validación, límites y registro. El modelo aporta capacidad; el harness la convierte en un sistema controlable.

En la charla se mostró con ocho piezas dentro de un mismo borde, y el modelo de OpenAI o Anthropic queda fuera, al que se llama por API:

Pieza	Qué hace
Modelo	Predice la siguiente acción o respuesta
Contexto	Instrucciones y datos que recibe en cada paso
Agent loop	El ciclo de pedir, actuar y observar
Memoria	El historial de mensajes entre pasos o conversaciones
Herramientas	Funciones de Python que el modelo puede pedir
Validación	Que la salida cumpla el contrato de tipos
Límites	Plazos y reintentos para que nada corra sin tope
Registro	Trazas para ver qué pasó en cada ejecución

Un matiz importante: el harness no equivale a todo el producto, ni garantiza que el contenido de una respuesta sea verdadero. Pydantic AI aporta piezas del harness, no toda la arquitectura de producción.

08 — FastAPI + agente: una petición de punta a punta

El recorrido de una petición es: el cliente llama a FastAPI, FastAPI valida y llama al agente, el agente usa una herramienta que consulta Postgres y devuelve una salida validada. El endpoint se ve así:

```
@app.post("/triage")
async def triage(ticket: Ticket, request: Request) -> Triage:
    deps = Deps(db=request.app.state.db)
    async with request.app.state.slots, asyncio.timeout(deadline):
        result = await agent.run(ticket.text, deps=deps, model=model)
    return result.output
```

Qué ocurre en cada línea:

- `ticket: Ticket` hace que FastAPI valide la petición con Pydantic antes de ejecutar nada. Una petición mal formada ni siquiera llega al agente.
- `-> Triage` declara el tipo de respuesta del endpoint: la misma clase que usa el agente como salida.
- `deps = Deps(db=...)` arma las dependencias tipadas con la conexión que vive en el estado de la aplicación.
- `request.app.state.slots` es un semáforo que limita cuántos agentes corren a la vez. Sin tope, una ráfaga de peticiones puede saturar el servicio o la cuenta del proveedor.
- `asyncio.timeout(deadline)` pone un plazo total desde fuera. Los límites de uso de Pydantic AI no son un plazo: el tiempo se controla con Python.
- `agent.run(...)` ejecuta el agent loop y `result.output` devuelve la salida ya validada.

09 — Producción: operar con control bajo carga

Escalar un agente es operar con control bajo carga. La charla resume cuatro controles:

1. **Pruebas y evaluaciones.** Una prueba comprueba que el código hace lo que dice. Una evaluación mide la calidad de las respuestas con casos conocidos. Se necesitan las dos y antes de cambiar el agente, no después.

2. **Límites y plazos.** Cuánto puede tardar una ejecución, cuántas pueden correr a la vez y cuántos reintentos se permiten.
3. **Trazas.** Ver cada ejecución, cada herramienta llamada y cada validación. Sin trazas, un fallo es una anécdota; con trazas, es un dato.
4. **Servicio asíncrono.** Un servicio que atiende muchas peticiones sin bloquearse y con la concurrencia acotada.

Un agente en producción necesita pruebas, límites y trazas, no solo un buen prompt. Los sistemas multiagente y los grafos de ejecución quedan fuera de esta guía y se dejan como lectura posterior.

10 — Agentes de dominio: mi mentalidad

La mentalidad que defiende la charla se dice en una línea: **no diseñes un agente para todo, diseña uno excelente para algo.**

Un agente de dominio hace un trabajo concreto. Recibe un contexto acotado, usa herramientas acotadas, respeta una política local y, cuando le falta información, pasa a revisión humana en lugar de adivinar. Un agente genérico intenta hacer de todo y por eso es difícil saber cuándo falla.

La referencia es la charla de Justin Schroeder (Standard Agents), «The Future Is Domain-Specific Agents», presentada en el AI Engineer World's Fair 2026. Es una visión, no un benchmark: no demuestra que un agente de dominio sea mejor en todos los casos.

11 — Por qué importan los agentes de dominio

Tres razones prácticas:

- **Se pueden evaluar.** Un trabajo concreto tiene entradas y salidas esperadas. Puedes juntar casos conocidos y medir si el agente lo hizo bien. Con un agente que «hace de todo», no sabes qué medir.
- **Se pueden acotar.** Si el contexto, las herramientas y la política son limitados, se reduce lo que puede salir mal. Un agente que solo puede consultar el plan de un cliente no puede borrar la base de datos.
- **Se pueden operar.** Con límites, trazas y revisión humana cuando falta información, el agente crece con control: sabes cuánto cuesta, cuánto tarda y cuándo pide ayuda.

12 — La demostración: un trabajo concreto

La demostración cierra la parte técnica con un agente de dominio resolviendo un trabajo real, con datos sintéticos. Para seguirla conviene mirar tres cosas:

1. **La entrada.** Qué recibe el agente y qué contexto se le da.
2. **La herramienta.** Qué función llama y con qué argumentos.
3. **La salida validada.** Qué forma tiene la respuesta y qué pasa cuando no se puede decidir.

13 — Las tres ideas que deben permanecer

1. **Los tipos son el contrato.** Pydantic valida la petición, los argumentos de las herramientas y la salida del agente.
2. **El harness hace fiable al modelo.** Contexto, validación, límites y registro rodean al modelo.
3. **Un agente, un trabajo.** Con contexto y herramientas acotados, el resultado se puede evaluar.

Una advertencia que conviene repetir: validar la forma de una salida no prueba que sea verdadera. Una respuesta puede cumplir el contrato de tipos y estar equivocada. Para eso existen las pruebas, las

evaluaciones y la revisión.

14 — Glosario mínimo

Término	Significado
Agente	Un modelo que elige qué herramienta usar hasta tener una respuesta
Agent loop	El ciclo de decidir, actuar con una herramienta y volver a decidir
Harness	El sistema que rodea al modelo: contexto, herramientas, validación, límites y registro
Herramienta (tool)	Una función de Python que el modelo puede pedir ejecutar
Dependencias (deps)	Lo que una herramienta necesita para trabajar, como la conexión a la base de datos, recibido con tipos
`output_type`	El tipo que describe la forma de la respuesta del agente
Agente de dominio	Un agente diseñado para un trabajo concreto, con contexto y herramientas acotados
Traza	El registro de una ejecución: qué se llamó, con qué datos y qué devolvió
Evaluación	Una medición de calidad con casos conocidos

15 — Cómo practicar: cuatro pasos

1. **Lee los recursos.** Empieza por la documentación de Pydantic AI y las lecturas de esta guía. Con una hora de lectura alcanza para entender el modelo mental.
2. **Elige un dominio.** Escoge un trabajo concreto que conozcas bien: clasificar tickets, extraer datos de documentos, responder preguntas sobre un manual. Define su contexto, sus herramientas y cómo sabrás que salió bien.
3. **Mira lo que ocurre.** Activa trazas y revisa cada ejecución. Se aprende más de diez trazas reales que de cualquier teoría.
4. **Prueba.** Escribe una prueba de tu agente con `TestModel`, el modelo de pruebas de Pydantic AI: verifica la salida y la lógica sin llamar a un modelo real, sin red y sin costo.

16 — Cursos sugeridos para profundizar

Escogí cursos que se pueden empezar gratis y que cubren partes distintas del camino. Verifiqué sus páginas el 22 de octubre de 2026; precios y contenidos pueden cambiar.

Curso	Qué aprenderás	Nivel y costo
Pydantic for LLM Workflows (DeepLearning.AI, Ryan Keenan)	Modelos de Pydantic, validación de respuestas de modelos, pasar un modelo en una llamada a la API y llamadas a herramientas, con un proyecto de asistente de soporte. Es la base de «los tipos son el contrato».	Intermedio, unas 2 horas. Gratis para inscribirse; el reconocimiento requiere membresía Pro.
Agentic AI (DeepLearning.AI, Andrew Ng)	Flujos agénticos, reflexión, uso de herramientas y MCP, evaluaciones y análisis de errores, y patrones de agentes muy autónomos. Da el marco conceptual para decidir cuánta autonomía darle a un agente.	Intermedio, unas 10 horas. Gratis para auditar; el certificado requiere membresía Pro.
AI Agents Course (Hugging Face)	Teoría, diseño y práctica de agentes con smolagents, LlamaIndex y LangGraph, y unidades extra de observabilidad y evaluación. Sirve para comparar enfoques de otros frameworks con el de Pydantic AI.	Python básico. Gratis, con dos certificados gratuitos; unas cinco semanas a ritmo de 3 a 4 horas por semana.
AI Agents for Beginners (Microsoft, GitHub)	18 lecciones con código para empezar a construir agentes. Una alternativa práctica si prefieres leer y ejecutar código a ver videos.	Principiante. Gratis, en GitHub.

17 — Lecturas y documentación recomendadas

Cada recurso con lo que vas a encontrar y para qué sirve:

- **Documentación de Pydantic AI.** El punto de partida. Explica el agente, las herramientas y la salida estructurada. Lee primero la página de salida (`output_type`), que es el contrato de la charla.
- **Pruebas unitarias con Pydantic AI.** Cómo usar `TestModel` y `override` para probar un agente sin llamar al modelo. Es el cuarto paso de la sección anterior.
- **Integración con Logfire.** Cómo ver trazas de cada ejecución de un agente. Útil para el tercer paso.
- **Tutorial de FastAPI.** Si todavía no conoces FastAPI, este tutorial es la mejor manera de entender la mitad de la charla: validación con Pydantic, rutas y dependencias.
- **Coroutines y tareas de Python.** Explica `asyncio.timeout` y las tareas asíncronas que usa el endpoint de la charla.
- **«Building effective agents» (Anthropic).** Un texto que distingue flujos de trabajo de agentes y presenta patrones como encadenar prompts, enrutar y orquestadores con trabajadores. Su consejo central: empieza por la solución más simple que funcione y agrega complejidad solo cuando mejora algo medible.
- **«Pydantic AI v1» (Samuel Colvin).** El anuncio de la versión 1: qué promete el framework y por qué le importa la estabilidad.
- **«The Future Is Domain-Specific Agents» (Justin Schroeder).** La charla que inspira la parte de agentes de dominio.

18 — Temas para profundizar

- **Historial en PostgreSQL.** Guardar la conversación de un agente en una columna `jsonb` para continuar donde se quedó.

- **Búsqueda vectorial con pgvector.** Dar al agente una herramienta de recuperación para que responda con datos de tu propio contenido.
- **Evaluaciones.** Medir la calidad con casos conocidos antes de cambiar el modelo, las instrucciones o las herramientas.
- **Trazas con Logfire.** Ver cada ejecución, cada herramienta y cada validación.
- **Reintentos con `ModelRetry`.** Qué hacer cuando una validación de salida falla: pedirle al modelo que lo intente de nuevo con el error a la vista.
- **Plazos y concurrencia.** Qué pasa cuando muchas peticiones llegan a la vez y cómo ponerles un tope.

19 — Fuentes y límites de lo dicho

- El código mostrado es de ejemplo, con datos sintéticos, y se probó con Pydantic AI 2.56.0 al prepararlo. La API de Pydantic AI cambia entre versiones: revisa la documentación de la versión que instales. Los ejemplos usan `output_type`, no la API antigua `result_type`.
- Los sistemas multiagente y los grafos de ejecución no se cubrieron.
- La visión de agentes de dominio es una postura, respaldada por una charla, no por un estudio comparativo.
- Los logos y nombres de herramientas pertenecen a sus titulares y se mencionan solo como referencia.

20 — Sobre el expositor

Ricardo Tovar. AI Engineering en formación en la Universidad Tecnológica de Panamá y cofundador de MART Automations. Su interés es llevar agentes de la demostración a producción con la menor magia posible.

- GitHub: github.com/rtovardev
- LinkedIn: linkedin.com/in/ricardotovar-ai
- Instagram: instagram.com/ricardotovar.ai
- Tarjeta digital: ricardo-card.vercel.app